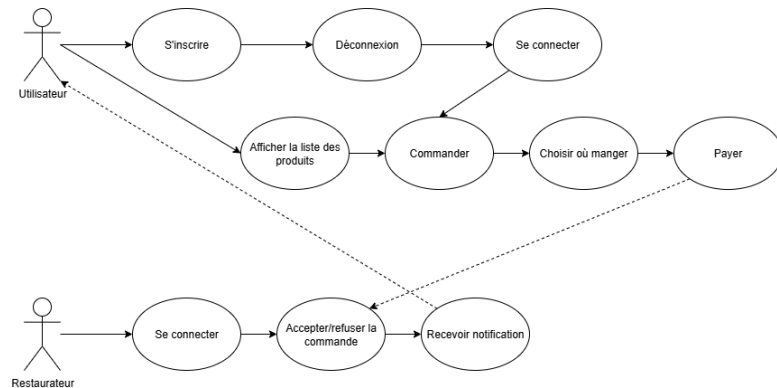


DESCRIPTION DU BESOIN

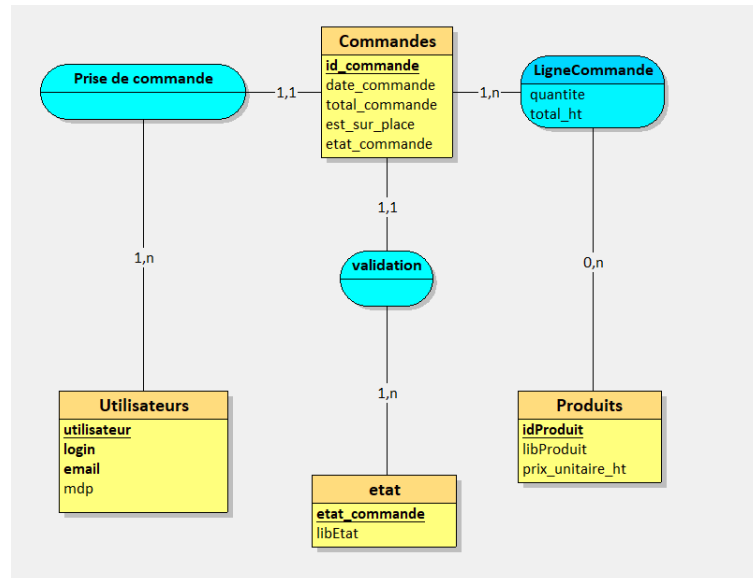
1) Documents de conception

1.1) Le diagramme des cas d'utilisation



Le système de commande implique deux acteurs principaux : l'Utilisateur (le client) et le Restaurateur. L'utilisateur peut interagir avec le système de plusieurs manières, notamment s'inscrire, se déconnecter, et se connecter pour accéder aux fonctionnalités principales. Une fois connecté, il peut afficher la liste des produits pour choisir et ensuite Commander. Le processus de commande se poursuit par Choisir où manger (livraison ou sur place/à emporter) et se termine par le Payer. Simultanément, le Restaurateur se connecte au système pour pouvoir accepter/refuser la commande reçue de l'utilisateur. Après cette décision, le restaurateur peut recevoir notification pour les mises à jour (l'utilisateur peut aussi en recevoir, comme l'indique la flèche en pointillés).

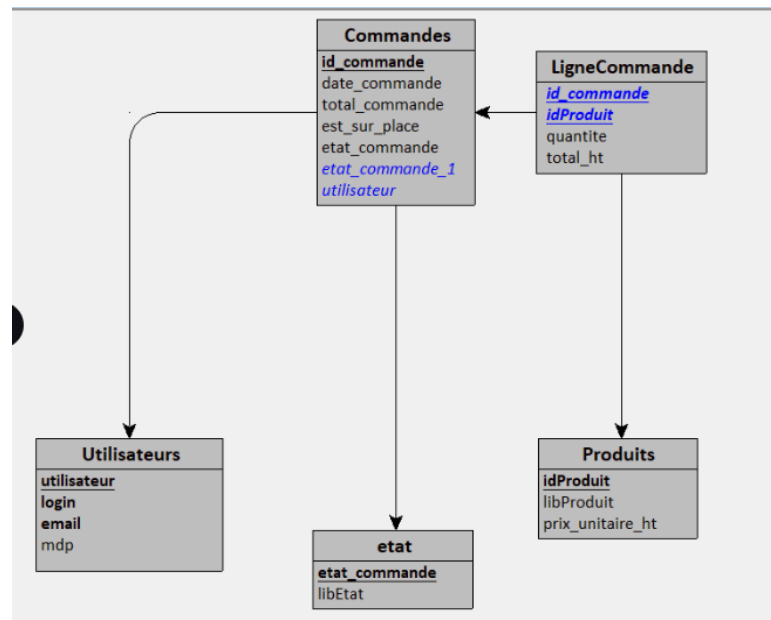
1.2) Le modèle conceptuel des données (looping ou équivalent)



Ce modèle organise les données autour de quatre entités principales (rectangles jaunes) : Commandes, LigneCommande, Utilisateurs, et Produits.

- Les utilisateurs peuvent réaliser une ou plusieurs Commandes (relation "Prise de commande", cardinalité 1,n).
- Chaque Commande est composée d'une ou plusieurs LigneCommande (cardinalité 1,n) qui détaillent la quantité et le prix total hors taxe par article.
- Chaque LigneCommande se réfère à un seul Produit, tandis qu'un Produit peut être inclus dans zéro à plusieurs LigneCommande (cardinalité 0,n).
- L'état actuel de chaque Commande est spécifié par la relation "validation" qui la lie à l'entité etat (cardinalité 1,n), où l'entité etat contient l'identifiant et le libellé de l'état (ex: en attente, prête, servie, etc.).

1.3) Le modèle logique des données (PHPMyAdmin ou équivalent)



Ce diagramme représente la structure relationnelle des tables (rectangles gris) et la façon dont elles sont liées par des clés primaires et étrangères (soulignées et en bleu).

- Tables Principales et Clés Primaires (soulignées) :
 - Utilisateurs (clé primaire : utilisateur)
 - Commandes (clé primaire : id_commande)
 - Produits (clé primaire : idProduit)
 - etat (clé primaire : etat_commande)
- Liaisons par Clés Étrangères (en bleu)
 - La table Commandes contient les clés étrangères utilisateur (qui lie la commande à l'utilisateur qui l'a passée) et etat_commande_1 (qui lie la commande à son état actuel dans la table etat).
 - La table LigneCommande représente la relation entre Commandes et Produits. Sa clé est composée (id_commande, idProduit) et elle

contient les clés étrangères `id_commande` (qui la lie à la commande)
et `idProduit` (qui la lie au produit commandé).

1.4) Le modèle physique des données (script(s) SQL)

```
-- phpMyAdmin SQL Dump
-- version 5.2.1
-- https://www.phpmyadmin.net/
--
-- Host: localhost
-- Generation Time: Nov 05, 2025 at 09:43 PM
-- Server version: 10.4.28-MariaDB
-- PHP Version: 8.2.4

SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
START TRANSACTION;
SET time_zone = "+00:00";

-- Création de la base de données
CREATE DATABASE IF NOT EXISTS RoseBlanche;
USE RoseBlanche;

/*!40101 SET
@OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET
@OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS
*/;
/*!40101 SET
@OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;

--
-- Database: `RoseBlanche`
--

--
-- -----
--
-- Table structure for table `Commandes`
--

CREATE TABLE `Commandes` (
  `idCommande` int(11) NOT NULL,
```

```
`date_commande` datetime DEFAULT NULL,  
`total_commande` decimal(10,2) DEFAULT NULL,  
`est_sur_place` tinyint(1) DEFAULT NULL,  
`idEtat` int(11) NOT NULL,  
`idUtilisateur` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;
```

```
-- -----
```

```
--  
-- Table structure for table `Etat`  
--
```

```
CREATE TABLE `Etat` (  
  `idEtat` int(11) NOT NULL,  
  `libEtat` varchar(50) DEFAULT NULL,  
  `descriptif` varchar(100) NOT NULL DEFAULT ''  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;
```

```
--  
-- Dumping data for table `Etat`  
--
```

```
INSERT INTO `Etat` (`idEtat`, `libEtat`, `descriptif`) VALUES  
(1, 'initialisée', 'le client est en train de choisir sa commande'),  
(2, 'finalisée', 'le choix de la commande est terminé et validé'),  
(3, 'calculée', 'le total de la commande est calculé et en attente de paiement'),  
(4, 'en attente', 'la commande est payée et en attente de préparation par le  
restaurateur'),  
(5, 'abandonnée', 'la commande est refusée par le restaurateur'),  
(6, 'en préparation', 'la commande est acceptée et en cours de préparation  
par le restaurateur'),  
(7, 'prête', 'la commande est prête et en attente d être récupérée par le  
client'),  
(8, 'servie', 'la commande a été récupérée par le client');
```

```
-- -----
```

```
--  
-- Table structure for table `LigneCommande`  
--
```

```

CREATE TABLE `LigneCommande` (
  `idCommande` int(11) NOT NULL,
  `idProduit` int(11) NOT NULL,
  `quantite` int(11) DEFAULT NULL,
  `total_ht` decimal(8,2) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;

--
-- Triggers `LigneCommande`
--
DELIMITER $$
CREATE TRIGGER `after_insert_lignecommande` AFTER INSERT ON
`LigneCommande` FOR EACH ROW BEGIN
  DECLARE v_total_ht DECIMAL(10, 2);
  DECLARE v_est_sur_place BOOLEAN;
  DECLARE v_tva DECIMAL(5, 3);

  SELECT SUM(total_ht) INTO v_total_ht FROM lignecommande
  WHERE idCommande = NEW.idCommande;

  SELECT est_sur_place INTO v_est_sur_place FROM commandes
  WHERE idCommande = NEW.idCommande;

  IF v_est_sur_place = 1 THEN
    SET v_tva = 1.1;
  ELSE
    SET v_tva = 1.055;
  END IF;

  UPDATE commandes
  SET total_commande = v_total_ht * v_tva
  WHERE idCommande = NEW.idCommande;

END
$$
DELIMITER ;
DELIMITER $$
CREATE TRIGGER `after_update_lignecommande` AFTER UPDATE ON
`LigneCommande` FOR EACH ROW BEGIN
  DECLARE v_total_ht DECIMAL(10, 2);
  DECLARE v_est_sur_place BOOLEAN;
  DECLARE v_tva DECIMAL(5, 3);

```

```
SELECT SUM(total_ht) INTO v_total_ht FROM lignecommande
WHERE idCommande = NEW.idCommande;
```

```
SELECT est_sur_place INTO v_est_sur_place FROM commandes
WHERE idCommande = NEW.idCommande;
```

```
IF v_est_sur_place = 1 THEN
    SET v_tva = 1.1;
ELSE
    SET v_tva = 1.055;
END IF;
```

```
UPDATE commandes
SET total_commande = v_total_ht * v_tva
WHERE idCommande = NEW.idCommande;
```

```
END
```

```
$$
```

```
DELIMITER ;
```

```
DELIMITER $$
```

```
CREATE TRIGGER `before_insert_lignecommande` BEFORE INSERT ON
`LigneCommande` FOR EACH ROW BEGIN
    DECLARE v_prix_ht DECIMAL(10,2);
```

```
    SELECT prix_unitaire_ht INTO v_prix_ht
    FROM produits
    WHERE idProduit = NEW.idProduit;
```

```
    SET NEW.total_ht = NEW.quantite * v_prix_ht;
```

```
END
```

```
$$
```

```
DELIMITER ;
```

```
DELIMITER $$
```

```
CREATE TRIGGER `before_update_lignecommande` BEFORE UPDATE ON
`LigneCommande` FOR EACH ROW BEGIN
    DECLARE v_prix_ht DECIMAL(10,2);
```

```
    SELECT prix_unitaire_ht INTO v_prix_ht
    FROM produits
    WHERE idProduit = NEW.idProduit;
```

```
    SET NEW.total_ht = NEW.quantite * v_prix_ht;
```

```

END
$$
DELIMITER ;

-- -----

--
-- Table structure for table `Produits`
--

CREATE TABLE `Produits` (
  `idProduit` int(11) NOT NULL,
  `libProduit` varchar(50) DEFAULT NULL,
  `prix_unitaire_ht` decimal(10,2) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;

--
-- Dumping data for table `Produits`
--

INSERT INTO `Produits` (`idProduit`, `libProduit`, `prix_unitaire_ht`) VALUES
(1, 'le simple cheese', 9.99),
(2, 'le double cheese', 11.99),
(3, 'le triple cheese', 13.99),
(4, 'le XXL triple monstre', 17.99);

-- -----

--
-- Table structure for table `Utilisateurs`
--

CREATE TABLE `Utilisateurs` (
  `idUtilisateur` int(11) NOT NULL,
  `login` varchar(50) NOT NULL,
  `email` varchar(100) NOT NULL,
  `mdpUtilisateur` varchar(300) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;

--
-- Indexes for dumped tables
--

```



```

--
-- Indexes for table `Commandes`
--
ALTER TABLE `Commandes`
  ADD PRIMARY KEY (`idCommande`),
  ADD KEY `idEtat` (`idEtat`),
  ADD KEY `idUtilisateur` (`idUtilisateur`);

--
-- Indexes for table `Etat`
--
ALTER TABLE `Etat`
  ADD PRIMARY KEY (`idEtat`);

--
-- Indexes for table `LigneCommande`
--
ALTER TABLE `LigneCommande`
  ADD PRIMARY KEY (`idCommande`,`idProduit`),
  ADD KEY `idProduit` (`idProduit`);

--
-- Indexes for table `Produits`
--
ALTER TABLE `Produits`
  ADD PRIMARY KEY (`idProduit`);

--
-- Indexes for table `Utilisateurs`
--
ALTER TABLE `Utilisateurs`
  ADD PRIMARY KEY (`idUtilisateur`);

--
-- AUTO_INCREMENT for dumped tables
--

--
-- AUTO_INCREMENT for table `Commandes`
--
ALTER TABLE `Commandes`
  MODIFY `idCommande` int(11) NOT NULL AUTO_INCREMENT,
  AUTO_INCREMENT=64;

```

```

--
-- AUTO_INCREMENT for table `Etat`
--
ALTER TABLE `Etat`
  MODIFY `idEtat` int(11) NOT NULL AUTO_INCREMENT,
  AUTO_INCREMENT=9;

--
-- AUTO_INCREMENT for table `Produits`
--
ALTER TABLE `Produits`
  MODIFY `idProduit` int(11) NOT NULL AUTO_INCREMENT,
  AUTO_INCREMENT=5;

--
-- AUTO_INCREMENT for table `Utilisateurs`
--
ALTER TABLE `Utilisateurs`
  MODIFY `idUtilisateur` int(11) NOT NULL AUTO_INCREMENT,
  AUTO_INCREMENT=23;

--
-- Constraints for dumped tables
--

--
-- Constraints for table `Commandes`
--
ALTER TABLE `Commandes`
  ADD CONSTRAINT `commandes_ibfk_1` FOREIGN KEY (`idEtat`)
  REFERENCES `Etat` (`idEtat`),
  ADD CONSTRAINT `commandes_ibfk_2` FOREIGN KEY (`idUtilisateur`)
  REFERENCES `Utilisateurs` (`idUtilisateur`);

--
-- Constraints for table `LigneCommande`
--
ALTER TABLE `LigneCommande`
  ADD CONSTRAINT `lignecommande_ibfk_1` FOREIGN KEY
  (`idCommande`) REFERENCES `Commandes` (`idCommande`),
  ADD CONSTRAINT `lignecommande_ibfk_2` FOREIGN KEY (`idProduit`)
  REFERENCES `Produits` (`idProduit`);
COMMIT;

```

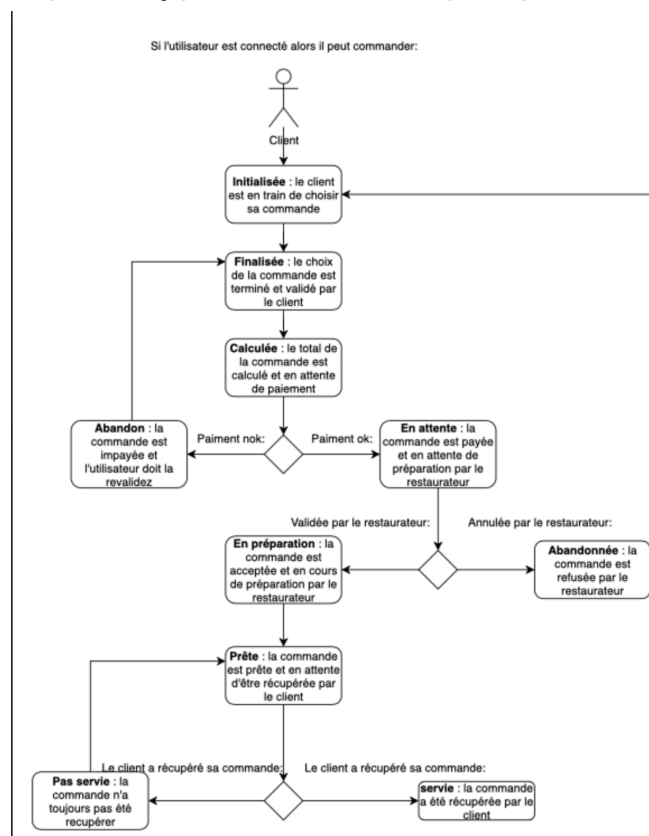
```
/*!40101 SET  
CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;  
/*!40101 SET  
CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;  
/*!40101 SET  
COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
```

1.5) Une description des valeurs possibles pour :

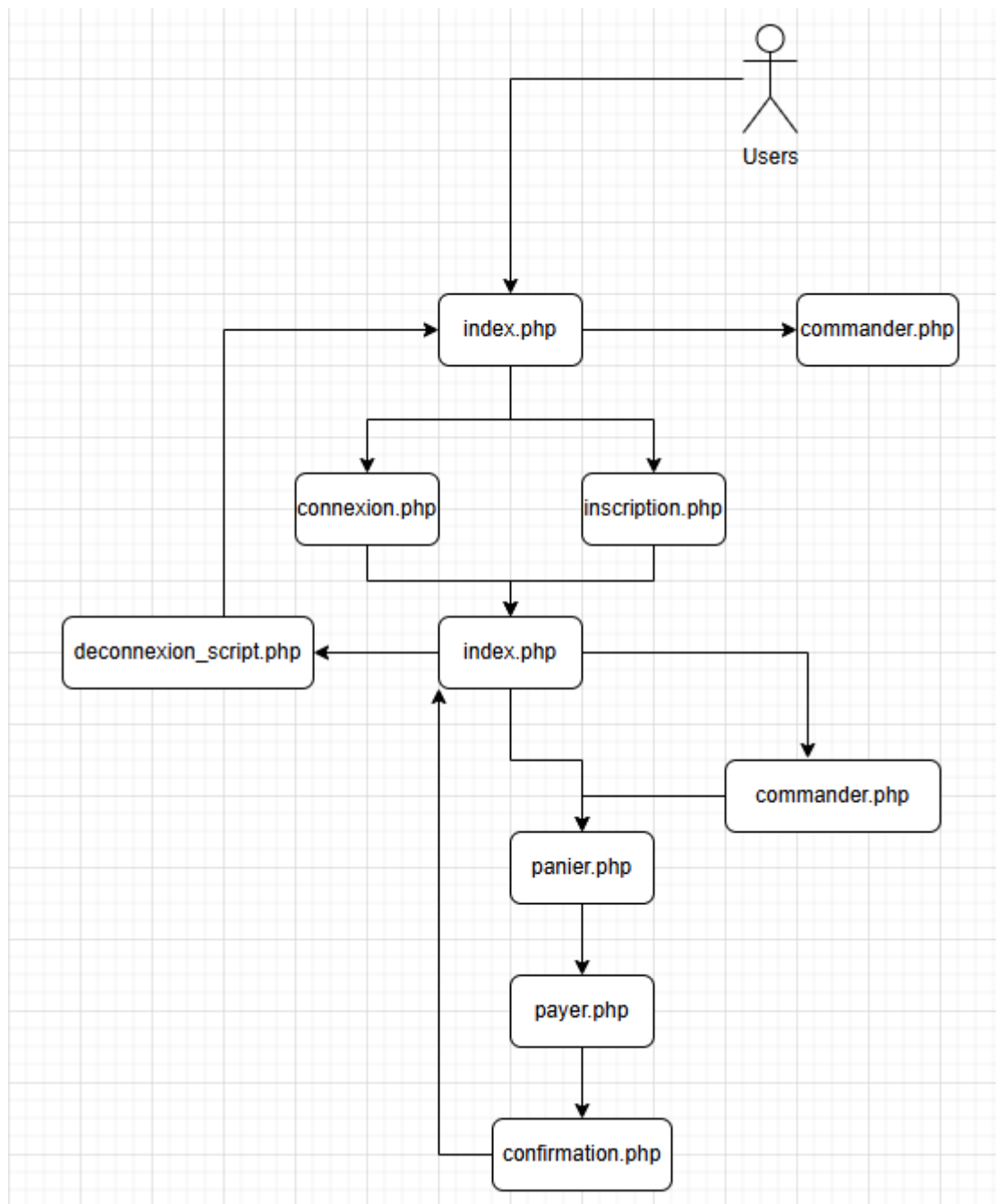
1.5.1) Les états des commandes

L'état d'une commande évolue selon son progression dans le système, allant de l'initialisation jusqu'à son service ou son abandon. Les valeurs possibles pour l'état d'une commande sont : Initialisée (le client choisit sa commande), Finalisée (le choix est terminé et validé), Calculée (le total est calculé et le système attend le paiement). Si le paiement échoue, l'état devient Abandon (la commande est impayée et doit être revalidée). Si le paiement réussit, l'état passe à En attente (la commande est payée et en attente de préparation). À partir de là, si elle est acceptée, l'état devient En préparation (la commande est en cours de préparation par le restaurateur). Si elle est refusée par le restaurateur, elle passe à Abandonnée (la commande est refusée par le restaurateur). Une fois la préparation terminée, l'état est Prête (la commande est prête et attend d'être récupérée par le client). Enfin, si le client récupère sa commande, l'état final est Servie ; si elle n'est pas récupérée, elle passe à Pas Servie.

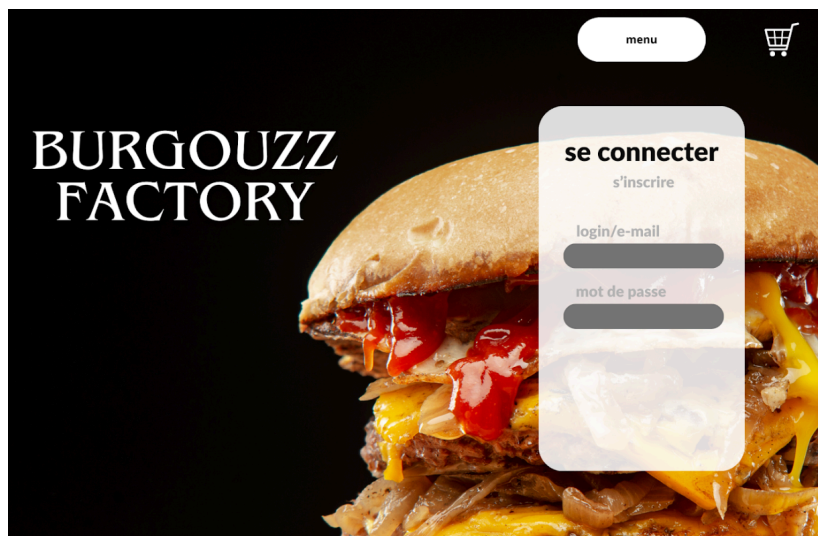
1.5.2) Les types de conso. (sur place/à emporter)

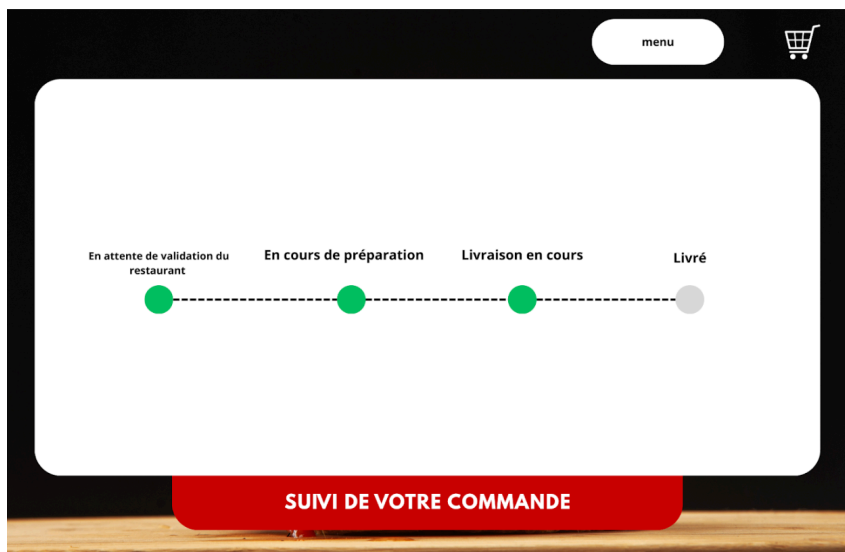


1.6) Le sitemap (enchaînement des pages)



1.7) Les maquettes de l'IHM (Balsamiq ou équivalent)





2) Documents de réalisation

2.1) Le manuel d'installation :

2.1.1) Les releases ou la branche git à utiliser

Pour cloner le dépôt :

```
git clone https://github.com/ton-compte/appResto-restoWeb.git  
cd appResto-restoWeb/appRoseBlanche
```

Configurer la base de données

importer le fichier mpd.sql directement (c'est un script de création et de peuplement) :

```
SOURCE appRoseBlanche/mpd.sql;
```

importer le fichier insert.sql directement (c'est un script de peuplement pour les jeux de tests):

```
SOURCE appRoseBlanche/insert.sql;
```

Lancer le serveur php:

```
php -S localhost:8000
```

Pour ouvrir dans le navigateur <http://localhost:8000>

2.2) Le manuel du jeu de test

2.2.1) Quel sont les user/password installés pour les tests

Identifiants de test

nom d'utilisateur : test

Mot de passe : test

fonction : Utilisateur de test pour vérifier que l'application tourne bien

nom d'utilisateur : testn2

mot de passe : test

fonction : Second utilisateur de test pour vérifier que les fonctions de commandes/paiement:confirmation marchent

2.2.2) Quels sont les données existantes pour les tests

```
INSÉRER DANS `Utilisateurs`(`idUtilisateur`,`se connecter`,`e-mail`,`utilisateur mdp`)VALEURS
(1,'Test','test@q1.com','$2y$10$zGc.J1VV19Zkr64FHiGv901RcDAi40xerF2Fhf7ecenHjlUMSPjyS'),
(2,'testn2','test@1im.fr','$2y$10$HLC0NRPyo/X5yYU1qZ6Ey.y1TucESMg0IV.4E3X4bc5oa0bAmb8s6');

--
--Exportation des données pour la table `LigneCommande`
--

INSÉRER DANS `LigneCommande`(`idCommande`,`idProduit`,`quantite`,`total_ht`)VALEURS
(61,3,2,27.98);

--
--Vidage des données pour la table `Commandes`
--
[
INSÉRER DANS `Commandes`(`idCommande`,`date_commande`,`commande_totale`,`est_sur_place`,`état d'identification`,`idUtilisateur`)VALEURS
(61,'2025-10-20 13:55:40',30.78,1,4,1);
```

Pour les identifiants de tests:

Identifiants de test

Nom utilisateur	Mot de passe	Fonction
Test	test	Utilisateur de test pour vérifier que l'application tourne bien.
testn2	test	Second utilisateur de test pour vérifier que les fonctions de commandes/paiement/confirmation marchent.